

Designing Machine Learning Systems

MLOps Deep Dive — Part 2: Governance, Responsible AI, Cost, Memory Boosters & Active Recall

Companion to Part 1 (v3) — by Chip Huyen

6. MLOps — Continued from Part 1

6.5 Model Governance & Lineage Tracking

Model Governance is the set of policies, processes, and tools that ensure ML models are developed, deployed, and monitored in a controlled, auditable, and compliant manner. It answers: *Who built this model? On what data? When was it deployed? Who approved it?*

Key Components of Model Governance

Component	Description	Tools
Model Registry	Central catalogue of all models with versions, metadata, and stage (Staging/Production/Archived)	MLflow Registry, SageMaker Registry, Vertex AI Model Registry
Model Lineage	Full audit trail: which dataset → which features → which training run → which model version → which deployment	MLflow, DVC, Neptune, Weights & Biases
Data Lineage	Track data from source to feature to training set — who touched it, when, and how it was transformed	Apache Atlas, OpenLineage, DataHub, Marquez
Approval Workflows	Human-in-the-loop gates before production deployment — sign-off from data science lead, risk team, legal	Custom workflows in Jira, GitHub PRs, SageMaker Approval
Access Control	Role-based access to models, datasets, and pipelines (who can train, deploy, or delete)	AWS IAM, GCP IAM, Azure RBAC, Vault
Audit Logs	Immutable record of all actions on models and data — required for regulatory compliance (GDPR, HIPAA, SOX)	AWS CloudTrail, GCP Audit Logs, Azure Monitor

Why Lineage Matters: If a model makes a wrong prediction that causes financial loss or harm, you need to trace back: Was it bad training data? A bug in feature engineering? A stale model? Without lineage, debugging production failures is nearly impossible.

6.6 Data Contracts & Schema Validation

A **data contract** is a formal agreement between data producers (upstream teams) and data consumers (ML pipelines) about the structure, quality, and semantics of data. It prevents silent data failures — the #1 cause of model degradation in production.

Data Contract Components

- **Schema:** Column names, data types, nullable constraints
- **Semantics:** What does each field mean? What are valid values?
- **SLAs:** Data freshness guarantees (e.g., "user_events table updated every 15 minutes")
- **Quality rules:** No nulls in user_id, age must be between 0–120, revenue must be non-negative
- **Ownership:** Who is responsible for this data? Who to contact when it breaks?

Schema Validation Tools

Tool	What It Does	Best For
Great Expectations	Define and run data quality "expectations" as code; generates data docs	Batch data validation in pipelines
Deequ (AWS)	Data quality checks on Spark DataFrames at scale	Large-scale Spark pipelines
Pandera	Pandas DataFrame schema validation with Python type hints	Lightweight validation in notebooks/scripts
Soda Core	SQL-based data quality checks; integrates with dbt	Data warehouse validation
Monte Carlo	Automated data observability — detects anomalies without writing rules	Enterprise data reliability
dbt Tests	Built-in schema tests (not_null, unique, accepted_values, relationships)	SQL transform pipelines

Watch Out: Schema drift is silent and deadly. A column renamed upstream, a new null value introduced, or a unit change (dollars to cents) can silently corrupt your model's features without any error being thrown. Always validate data at pipeline entry points.

6.7 Model Cards & Documentation

A **Model Card** (introduced by Google) is a short document that accompanies a trained ML model, providing transparency about its intended use, performance, limitations, and ethical considerations. Think of it as a "nutrition label" for ML models.

Model Card Structure

Section	What to Include
Model Details	Model name, version, type, training date, authors, contact

Intended Use	Primary use cases, out-of-scope uses, intended users
Training Data	Dataset description, size, collection method, preprocessing steps, known biases
Evaluation Data	Test set description, how it differs from training data
Performance Metrics	Overall metrics + slice-based metrics (by gender, age, geography, etc.)
Ethical Considerations	Potential harms, fairness analysis, privacy implications
Caveats & Recommendations	Known limitations, when NOT to use this model, recommended safeguards

Best Practice: Model cards should be living documents — updated every time the model is retrained or deployed to a new context. Store them in the model registry alongside the model artefact.

6.8 Responsible AI & Bias Auditing in MLOps

Responsible AI means building ML systems that are **fair, transparent, accountable, and privacy-preserving**. In MLOps, this means embedding fairness checks and bias audits directly into the CI/CD pipeline — not as an afterthought.

Types of Bias in ML Systems

Bias Type	Description	Example
Historical Bias	Training data reflects past societal inequalities	Hiring model trained on historical data that favoured men
Representation Bias	Certain groups underrepresented in training data	Facial recognition trained mostly on light-skinned faces
Measurement Bias	Proxy features that correlate with protected attributes	Using zip code as a proxy for race in credit scoring
Aggregation Bias	One model applied to groups with different underlying distributions	Single diabetes model for all ethnicities despite different risk profiles
Deployment Bias	Model used in a context different from its training context	Recidivism model trained on one state applied nationwide
Feedback Loop Bias	Model predictions influence future training data	Predictive policing model that reinforces over-policing in certain areas

Fairness Metrics

Metric	Definition	When to Use
	Equal positive prediction rates across groups	Hiring, loan approval

Demographic Parity		
Equal Opportunity	Equal true positive rates across groups	Medical diagnosis, parole decisions
Equalised Odds	Equal TPR and FPR across groups	High-stakes decisions with asymmetric costs
Individual Fairness	Similar individuals get similar predictions	Personalised recommendations
Calibration	Predicted probabilities match actual outcomes across groups	Risk scoring, insurance

Bias Auditing Tools

- **Fairlearn (Microsoft):** Fairness metrics + mitigation algorithms (reweighting, threshold optimisation)
- **AI Fairness 360 (IBM):** 70+ fairness metrics and 10+ bias mitigation algorithms
- **What-If Tool (Google):** Interactive visual exploration of model behaviour across subgroups
- **Aequitas:** Bias audit toolkit for classification models
- **SHAP + Slice-based evaluation:** Understand feature importance per demographic group

Explainability & Interpretability

Method	Type	Description
SHAP	Post-hoc, model-agnostic	Shapley values — how much each feature contributed to a prediction
LIME	Post-hoc, model-agnostic	Local linear approximation around a single prediction
Integrated Gradients	Post-hoc, gradient-based	Attribution for deep learning models (images, text)
Attention Maps	Built-in (Transformers)	Which tokens the model attended to
Decision Trees / Rules	Intrinsic	Inherently interpretable models
Partial Dependence Plots	Post-hoc	Marginal effect of one feature on predictions

6.9 Privacy in ML Systems

- **Differential Privacy:** Add calibrated noise to training data or gradients so individual records cannot be inferred (used by Apple, Google)
- **Federated Learning:** Train on data that never leaves the device — model updates (gradients) are aggregated centrally (used by Google Keyboard, healthcare)

- **Data Anonymisation:** Remove PII before training — k-anonymity, l-diversity, t-closeness
- **Secure Multi-Party Computation:** Multiple parties jointly train a model without sharing raw data
- **Model Inversion Attacks:** Adversaries can reconstruct training data from model outputs — mitigate with DP and output perturbation
- **Membership Inference Attacks:** Determine if a specific record was in the training set — mitigate with regularisation and DP

6.10 Cost Optimisation in MLOps

ML systems can be extremely expensive. Cost optimisation is a first-class concern in production MLOps.

Cost Drivers in ML Systems

Stage	Cost Driver	Optimisation Strategy
Data Storage	Storing raw + processed data in S3/GCS	Tiered storage (S3 Intelligent-Tiering), compression (Parquet/ORC), data lifecycle policies
Data Processing	Spark cluster compute hours	Spot/preemptible instances, right-size clusters, cache intermediate results
Model Training	GPU hours (A100s are expensive)	Spot instances, mixed precision (FP16), gradient checkpointing, early stopping, smaller batch sizes
Hyperparameter Tuning	Many training runs	Bayesian optimisation (fewer runs), early stopping (Hyperband), population-based training
Model Serving	Always-on GPU inference servers	Model compression (quantisation, pruning), batching requests, serverless for low-traffic, edge deployment
Feature Store	Online store (Redis) is expensive	Cache only high-frequency features; use TTL to expire stale features
Monitoring	Storing all predictions + features	Sample-based logging (log 1% of traffic), aggregate statistics instead of raw data

Model Compression Techniques (Cost vs Accuracy Trade-off)

Technique	How It Works	Size Reduction	Accuracy Loss
Quantisation (INT8)	Reduce weight precision from FP32 to INT8	4x smaller	~1-2%
Pruning	Remove near-zero weights from the network	2-10x smaller	1-5%
Knowledge Distillation	Train small "student" model to mimic large "teacher"	10-100x smaller	2-10%

Low-Rank Factorisation	Decompose weight matrices into smaller matrices	2-5x smaller	1-3%
ONNX Export	Convert to optimised runtime format	Same size, faster inference	None

6.11 MLOps for LLMs (Large Language Models)

LLMs introduce unique MLOps challenges beyond traditional ML systems:

Challenge	Traditional ML	LLMs
Model Size	MBs to GBs	GBs to TBs (GPT-4 ~1.8TB)
Training Cost	Hours on 1-8 GPUs	Months on thousands of GPUs (\$millions)
Serving Latency	ms with standard hardware	Requires GPU clusters; token generation is sequential
Evaluation	Standard metrics (AUC, F1)	Human eval, LLM-as-judge, BLEU/ROUGE (limited)
Adaptation	Full retrain or fine-tune	Prompt engineering, RAG, LoRA fine-tuning, RLHF
Monitoring	Feature drift, prediction drift	Hallucination rate, toxicity, latency per token, cost per query
Versioning	Model weights + code	Model weights + prompts + RAG knowledge base + system prompt

LLM-Specific MLOps Tools

- **LangSmith:** Tracing, evaluation, and monitoring for LangChain applications
- **Weights & Biases Prompts:** Prompt versioning and evaluation
- **Arize Phoenix:** LLM observability — hallucination detection, retrieval quality
- **vLLM:** High-throughput LLM serving with PagedAttention
- **TGI (Text Generation Inference):** HuggingFace's production LLM server
- **LoRA / QLoRA:** Parameter-efficient fine-tuning — train only a small adapter, not the full model

7. 🧠 Memory Boosting Layer — MLOps Edition

7.1 Powerful Analogies & Mental Models

✈️ Analogy 1: The Airline Industry

MLOps is like running an airline. The **ML model** is the aircraft. But the airline also needs **air traffic control (orchestration)**, **maintenance crews (monitoring)**, **flight logs (lineage tracking)**, **safety inspections (bias auditing)**, **fuel management (cost optimisation)**, and **passenger manifests (data contracts)**. A great aircraft with no maintenance crew will eventually crash. MLOps is the entire airline operation — not just the plane.

🏗️ Analogy 2: The Construction Site

MLOps Maturity Levels are like building construction stages. Level 0 = a person building a hut by hand with no plan. Level 1 = a small crew with blueprints. Level 2 = a professional construction company with project management, safety inspections, and quality control. Level 3 = a fully automated smart factory that builds houses robotically with zero defects. Most companies are at Level 1 trying to reach Level 2.

🧬 Analogy 3: The Hospital Lab

Data contracts and schema validation are like lab test standards in a hospital. Every blood sample must be collected in the right tube, labelled correctly, stored at the right temperature, and processed within a time window. If any step is violated, the test result is invalid — even if the lab equipment is perfect. Similarly, even the best ML model produces garbage if the input data violates the contract.

⚖️ Analogy 4: The Court of Law

Model governance and model cards are like legal documentation in a court case. Every decision must be traceable, documented, and defensible. "The model said so" is not acceptable in court — you need to show: what data was used, who approved the model, what its known limitations are, and whether it was tested for fairness. Model cards are your legal brief for the model.

🌱 Analogy 5: The Farm

Continual learning and CT (Continuous Training) are like farming. You don't plant seeds once and expect food forever. You plant (train), water (monitor), harvest (deploy), check soil quality

(drift detection), and replant each season (retrain). A farm that stops replanting eventually produces nothing. An ML system that stops retraining eventually becomes useless as the world changes around it.

7.2 Teach It to a 10-Year-Old 🧒

Imagine you taught your robot friend to sort your toys. That's the ML model. But MLOps is everything else you need to keep the robot working well:

1. **Write down the rules** — "sort red toys in box A, blue in box B" (data contracts)
2. **Keep a diary** — "on Monday I taught the robot this, on Tuesday I added new toys" (lineage tracking)
3. **Check if it's being fair** — "is the robot treating all toys equally, or ignoring some?" (bias auditing)
4. **Watch it work** — "is the robot still sorting correctly, or making mistakes?" (monitoring)
5. **Teach it again** — "you got new toys for your birthday, so teach the robot about them" (continual training)
6. **Save batteries** — "don't run the robot all day if you only need it for 1 hour" (cost optimisation)
7. **Write a report card** — "here's what the robot can do, what it can't do, and when to ask a grown-up instead" (model card)

MLOps is just making sure your robot toy-sorter keeps working well, is fair, doesn't waste batteries, and you always know what it's doing and why!

7.3 Quick Recap — The Space Mission Metaphor 🚀

Think of deploying an ML system as running a **space mission control centre**:

- 🚀 **The rocket (model)** is built once but needs constant monitoring after launch
- 📋 **Mission brief (model card)** — everyone knows what the mission is, what the rocket can do, and what to do if something goes wrong
- 📅 **Flight logs (lineage tracking)** — every manoeuvre is recorded; if something goes wrong, you can replay exactly what happened
- 🔍 **Pre-launch checklist (data contracts + schema validation)** — fuel type correct? Payload weight within limits? No missing components?
- 📡 **Mission control (monitoring)** — watching telemetry 24/7; alerts fire if trajectory deviates
- 🛠️ **Mid-flight corrections (continual learning)** — small thruster burns to correct course as conditions change

-  **Safety board (responsible AI)** — independent review to ensure the mission doesn't harm anyone
-  **Budget committee (cost optimisation)** — use the smallest rocket that can do the job; don't burn fuel unnecessarily
-  **Mission success = model still performing well 6 months after launch**

MLOps is mission control. Without it, your rocket launches beautifully — and then silently drifts off course into the void.

8. ? Active Recall — MLOps Edition

Questions

1. What are the 4 MLOps maturity levels? What is the key difference between Level 1 and Level 2?
2. What is the difference between CI, CD, and CT in the context of MLOps? What triggers each?
3. What is a data contract, and why is it critical for preventing silent ML failures?
4. What is model lineage, and why is it important for debugging production failures?
5. Name 3 types of bias in ML systems and give a real-world example of each.
6. What is the difference between demographic parity and equal opportunity as fairness metrics?
7. What is differential privacy, and how does it protect individual data in ML training?
8. Name 4 model compression techniques. Which one reduces model size the most with the least accuracy loss?
9. What are 3 unique MLOps challenges introduced by LLMs that don't exist in traditional ML?
10. What is a Model Card, and what 7 sections should it contain?

Answers

A1:

- **Level 0:** Manual — notebooks, no versioning, no monitoring, model deployed once
- **Level 1:** ML Pipeline Automation — automated training, experiment tracking, model registry, basic monitoring
- **Level 2:** CI/CD Automation — automated testing, automated deployment, drift-triggered retraining, full observability
- **Level 3:** Full Automation — self-healing, automated A/B testing, bandit deployment, real-time features

Key difference L1 vs L2: Level 1 automates training but deployment is still manual. Level 2 automates the entire pipeline including deployment and retraining — humans only intervene for governance approvals.

A2:

- **CI (Continuous Integration):** Triggered on every code commit. Runs linting, unit tests, data validation, and a smoke test of model training.
- **CD (Continuous Delivery):** Triggered after CI passes. Packages model into Docker, deploys to staging, runs integration tests, promotes to production (sometimes with manual approval).
- **CT (Continuous Training):** Triggered by schedule (daily/weekly), drift detection alert, or data volume threshold. Retrains the model on fresh data and auto-promotes if the challenger beats the champion.

A3: A data contract is a formal agreement between data producers and consumers specifying schema (column names, types), semantics (what values mean), quality rules (no nulls, valid ranges), SLAs (freshness guarantees), and ownership. It prevents silent failures — e.g., an upstream team renames a column or changes units, which silently corrupts ML features without throwing any error. Tools: Great Expectations, Pandera, dbt Tests, Soda Core.

A4: Model lineage is the complete audit trail connecting: raw data → feature engineering → training dataset → training run → model version → deployment. It's critical for debugging because when a model fails in production, you need to trace back: Was it bad data? A feature bug? A stale model? Without lineage, you're debugging blindly. Tools: MLflow, DVC, OpenLineage, DataHub.

A5:

- **Historical bias:** Hiring model trained on historical data that favoured men → perpetuates gender discrimination
- **Representation bias:** Facial recognition trained mostly on light-skinned faces → higher error rates for dark-skinned faces
- **Measurement bias:** Using zip code as a proxy for race in credit scoring → discriminates by neighbourhood

A6:

- **Demographic Parity:** The model must give positive predictions at equal rates across groups (e.g., 30% loan approval for both men and women). Ignores whether the predictions are correct.
- **Equal Opportunity:** The model must have equal True Positive Rates across groups (e.g., among people who actually qualify for a loan, men and women are approved at equal rates). Focuses on not missing qualified candidates from any group.

Equal Opportunity is generally preferred for high-stakes decisions because it focuses on not unfairly denying qualified individuals.

A7: Differential Privacy (DP) adds carefully calibrated mathematical noise to training data or model gradients so that the presence or absence of any single individual's data cannot be detected from the model's outputs. The privacy budget (epsilon ϵ) controls the trade-off: smaller ϵ = more privacy but less accuracy. Used by Apple (keyboard predictions), Google (Chrome telemetry), and US Census Bureau.

A8:

- **Quantisation (INT8):** 4x smaller, ~1-2% accuracy loss — best accuracy/size trade-off
- **Pruning:** 2-10x smaller, 1-5% accuracy loss
- **Knowledge Distillation:** 10-100x smaller, 2-10% accuracy loss — biggest size reduction

- **Low-Rank Factorisation:** 2-5x smaller, 1-3% accuracy loss

Best accuracy/size trade-off: Quantisation (INT8) — minimal accuracy loss with 4x size reduction, widely supported by hardware accelerators.

A9:

- **Model size:** LLMs are GBs to TBs — require GPU clusters just to load, unlike traditional models that fit on a single CPU
- **Evaluation:** No single metric captures LLM quality — requires human evaluation, LLM-as-judge, and task-specific benchmarks; traditional metrics like BLEU are insufficient
- **Versioning complexity:** Must version not just model weights but also prompts, system prompts, RAG knowledge bases, and fine-tuning adapters — all of which affect outputs

A10: A Model Card is a short transparency document accompanying a trained model. The 7 sections are:

1. **Model Details** — name, version, type, authors, date
2. **Intended Use** — primary use cases, out-of-scope uses
3. **Training Data** — dataset description, size, known biases
4. **Evaluation Data** — test set description
5. **Performance Metrics** — overall + slice-based metrics
6. **Ethical Considerations** — potential harms, fairness analysis
7. **Caveats & Recommendations** — limitations, when NOT to use

9. ⚡ Ultra-Short Summary

- 🔧 **MLOps = DevOps for ML** — automate the entire lifecycle: data → features → training → evaluation → deployment → monitoring → retraining.
- 📊 **MLOps Maturity**: Level 0 (manual notebooks) → Level 1 (automated training) → Level 2 (automated CI/CD/CT) → Level 3 (self-healing, bandit deployment). Most companies are at Level 1.
- 📄 **Data contracts + schema validation** prevent silent data failures — the #1 cause of model degradation in production.
- 📁 **Model lineage + governance** ensure every model decision is traceable, auditable, and defensible — critical for regulated industries.
- ⚖️ **Responsible AI** means embedding fairness audits, bias checks, explainability, and privacy (DP, federated learning) directly into the MLOps pipeline — not as an afterthought.
- 💰 **Cost optimisation** = spot instances for training + model compression (quantisation, distillation) for serving + sample-based logging for monitoring.

10. 🏛️ Complete MLOps Reference Card

MLOps Tool Stack — One-Page Reference

Category	Open Source	Managed/Cloud
Experiment Tracking	MLflow, DVC	W&B, Neptune, Comet
Pipeline Orchestration	Airflow, Prefect, Dagster	AWS Step Functions, Cloud Composer
ML Pipelines	Kubeflow, ZenML, Metaflow	SageMaker Pipelines, Vertex AI Pipelines
Feature Store	Feast, Hopsworks	Tecton, SageMaker FS, Vertex FS
Model Registry	MLflow Registry	SageMaker Registry, Vertex Model Registry
Data Versioning	DVC, LakeFS	Delta Lake, Apache Iceberg
Data Quality	Great Expectations, Pandera, dbt Tests	Monte Carlo, Soda Cloud, Deequ
Data Lineage	OpenLineage, Marquez	DataHub, Apache Atlas, Collibra

Model Serving	TorchServe, TF Serving, Triton, vLLM	SageMaker Endpoints, Vertex Endpoints
CI/CD	GitHub Actions, Jenkins, GitLab CI	AWS CodePipeline, Cloud Build
Containerisation	Docker, Podman	AWS ECR, GCR, ACR
Orchestration	Kubernetes, Helm, Kubeflow	EKS, GKE, AKS
Monitoring (Ops)	Prometheus, Grafana	CloudWatch, Datadog, New Relic
Monitoring (ML)	Evidently AI, WhyLogs	Arize, Fiddler, Arthur AI
Fairness / Bias	Fairlearn, AI Fairness 360, Aequitas	What-If Tool, SageMaker Clarify
Explainability	SHAP, LIME, Captum	SageMaker Debugger, Vertex Explainable AI
LLM Ops	LangSmith, Arize Phoenix, vLLM	W&B Prompts, Azure AI Studio
Cost Optimisation	Spot instances, ONNX, TFLite	SageMaker Savings Plans, Committed Use

MLOps Decision Framework

Question	Answer → Action
Is model performance degrading?	Check monitoring → identify drift type → trigger CT pipeline
Is data quality failing?	Check data contracts → fix upstream source → re-validate pipeline
Is serving too slow?	Profile inference → apply quantisation/pruning → consider edge deployment
Is training too expensive?	Use spot instances + mixed precision + early stopping + smaller architecture
Is the model biased?	Run fairness audit → apply mitigation (reweighting/threshold) → update model card
Is the model not reproducible?	Add DVC for data versioning + MLflow for experiment tracking + Docker for environment
Is deployment risky?	Use shadow → canary → A/B → bandit progression; never deploy directly to 100%
Is the team not aligned?	Create model cards + data contracts + shared model registry + approval workflows